OTel in Practice

How can I contribute to OpenTelemetry if I'm not a developer?



Diana Todea - Head of DevRel Engineering

VictoriaMetrics

About me



OTel Docs Approver

OTel Localization (Romanian maintainer, Spanish contributor)

CLOUD NATIVE COMPUTING FOUNDATION AMBASSADOR

TAG OPERATIONAL RESILIENCE

MERGE FORWARD CLOUD NATIVE COMPUTING FOUNDATION

KEPLER





GitHub: didiViking

Bluesky: @didiviking.bsky.social

X: @dianavtodea

Mastodon: @dianatodea

Instagram: @didivikingdevrel

Medium: @dianatodea

The myth: contribution = code

CODE ≠ **CONTRIBUTION**

OpenTelemetry explicitly lists docs, SIG participation, community work, localization, note-taking, events, and end-user experience as contribution paths.

What can a non-developer actually contribute?

Language

Translate · review terminology · maintain glossaries

Knowledge

Docs · examples · blog posts · case studies

Experience

End-user feedback · blueprints · reference implementations

Community

SIG notes · onboarding · events · contributor support

Quality

Spot gaps · reproduce problems · improve clarity

Direction

Join SIG discussions · bring real-world requirements

Localization is not “just translation”

LOCALIZATION = **language + technical accuracy + community context**

Translate meaning

Preserve the technical intent, not merely the words. Code, links and specific markup often stay untouched.

Review terminology

Decide how concepts such as span, baggage, context, exporter or sampling should read consistently.

Maintain over time

English docs evolve. Localized content needs updates, reviews, coordination and new contributors.

A real path: starting a Romanian localization

1

Find a gap

A language community is missing or incomplete.

2

Build a tiny team

A mentor + potential contributors.

3

Start with one page

The homepage becomes a concrete first milestone.

4

Create infrastructure

Language config, labels, approver group, ownership.

5

Grow the backlog

Small, reviewable pages become contributor entry points.

The contribution starts with language expertise — but quickly teaches GitHub workflow, review, governance and collaboration.

Your first contribution can be deliberately small

Find one page.

Translate or improve it.

Ask for review.

Ship.

What “good first issue” really gives you

A bounded task
A visible owner/community
Examples from previous PRs
A reason to ask questions
A review loop that teaches project norms

Small scope lowers risk for both the newcomer and the reviewer.

But GitHub still looks like code.



The technical workflow can become the barrier — even when the contribution itself is non-code.

AI can help you translate. It cannot own your contribution.

USE AI

Draft · suggest wording · explain terminology

→

VERIFY

Language · technical meaning · terminology · localization rules

→

OWN THE PR

Understand feedback · fix errors · improve the next contribution

THE TRAP — **Prompt → copy → PR** ✗

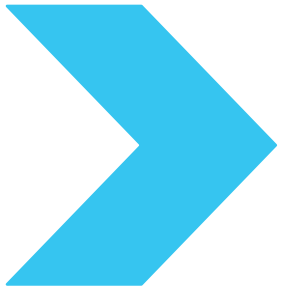
THE CONTRIBUTION — **Prompt → understand → verify → improve → PR** ✓

If you cannot review the translation yourself, you should not submit it.

The community is the abstraction layer

NEW CONTRIBUTOR

language expertise
user experience
curiosity
local context



COMMUNITY SUPPORT

clear issues • examples • Slack help
review feedback • mentors •
recognition

Good onboarding converts expertise into contribution.

Why do people come back after PR #1?



I learned something

Skill

Someone reviewed me

Belonging

My work is visible

Impact

I know what to do next

Momentum

Retention is not created by a README alone. It is created by the experience around the contribution.

What does the contributor gain?

Skills

GitHub workflow
Technical vocabulary
Writing and review
Open source governance
Cross-cultural communication

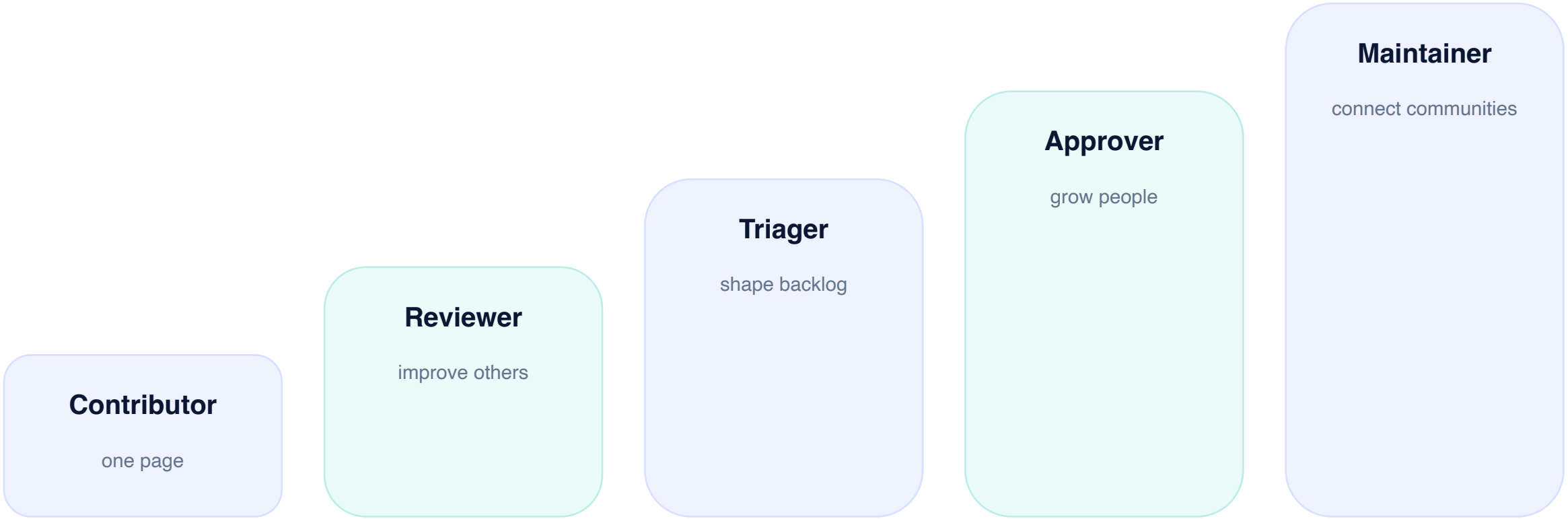
Network

Maintainers
Other translators
End users
SIG communities
Future collaborators

Agency

Influence clarity
Represent local users
Identify gaps
Own an area over time
Help the next newcomer

The path can grow without becoming “developer”



Growth can mean deeper community ownership — not a forced transition into coding.

What can OpenTelemetry — and other OSS projects — learn?

1	**Design for the first 30 minutes**	Make one useful task discoverable and bounded.
2	**Separate expertise from tooling**	Do not require GitHub mastery before someone can add value.
3	**Make review educational**	Explain decisions; teach the project's mental model.
4	**Create visible next steps**	A merged PR should open a path, not end one.
5	**Recognize non-code ownership**	Review, language stewardship and mentorship are real maintenance.

A 15-minute contribution menu

5 min	**Read a page as a newcomer**	Mark one confusing sentence, broken link or missing explanation.
10 min	**Join a SIG or localization channel**	Listen first. Ask where help is needed. Volunteer for notes or review.
15 min	**Take one bounded task**	A typo, terminology review, localization page, docs issue or end-user feedback.

You do not need permission to start learning the community.

Where to start in OpenTelemetry

Contributing	opentelemetry.io/docs/contributing
Localization	opentelemetry.io/docs/contributing/localization
Community + SIGs	opentelemetry.io/community
Slack	#otel-docs-localization · #otel-comms
End-user guidance	opentelemetry.io/docs/guidance

Pick the contribution path that uses a skill you already have.

Tips to help out scale the localization community

1. Create a guide for first time contributors
2. Set an example (open the PR, share tips and tricks, ask for reviews)
3. Be open to mentorship
4. Empower the other contributors

OpenTelemetry

https://opentelemetry.io/blog/2026/alternative-approaches-to-contributing/